

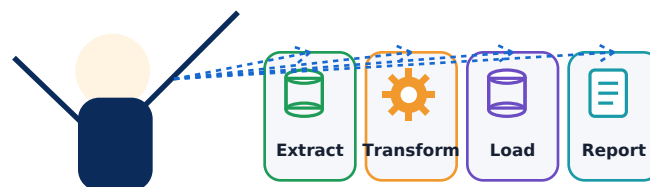


1 What Airflow is really doing

Apache Airflow is an open-source platform to author, schedule, and monitor workflows programmatically in Python. In data engineering, it usually acts as the control tower for recurring pipelines.

- Define pipelines as Python code
- Run on schedules or manual triggers
- Control task order using dependencies
- Track failures, retries, logs, and history

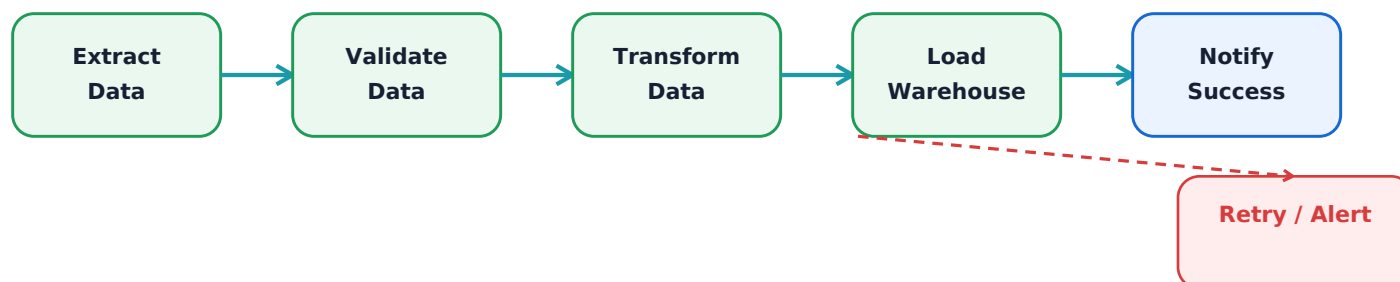
Airflow = conductor, not the orchestra



Conductor / Scheduler

2 The DAG idea

DAG = Directed Acyclic Graph. Directed means tasks have an order. Acyclic means no circular loops. Airflow uses this graph to decide what can run now and what must wait.



3 Core concepts at a glance



DAG

The full workflow definition - what runs, when it runs, and in what order.



Task

One unit of work inside a DAG. Each task has its own state and logs.



Operator

A task template: Python, Bash, SQL, Spark, Docker, Kubernetes, email, and more.



Scheduler

The brain that scans DAGs and decides which task instances are ready.



Worker

The process or machine that executes the actual task code.



Metadata DB

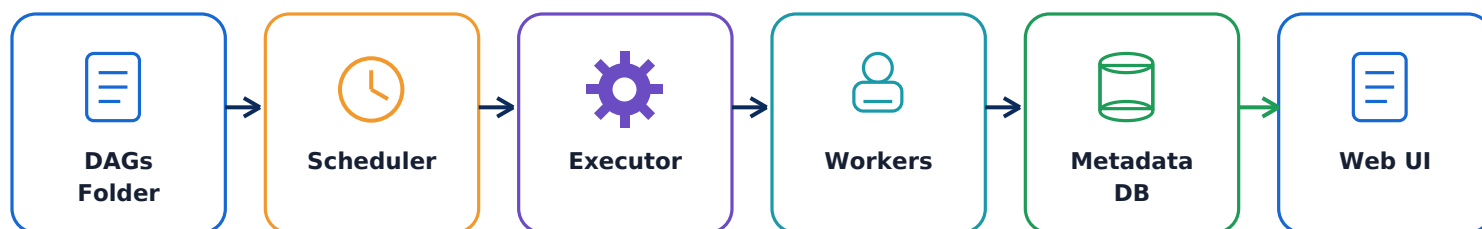
The memory of Airflow: DAG runs, task states, retries, logs, variables, XComs.

Airflow Architecture

How the pieces work together



4 Six key components



DAGs Folder

Directory of Python files. The scheduler scans it repeatedly. Keep files fast to parse.

Scheduler

Runs continuously, parses DAGs, checks schedules and dependencies, creates task instances.

Executor

Routes tasks. Local, Celery, and Kubernetes executors change how distributed the work is.

Workers

Run the actual task code. Workers may execute Python, SQL, shell, Spark submission, or API calls.

Metadata DB

Stores run history, task states, variables, connections, XComs, and scheduler decisions.

Web UI

Shows DAG graphs, task states, logs, retries, and operational history for troubleshooting.

5 Architecture data flow

DAG files define the pipeline shape. The scheduler reads those files, creates runnable task instances, and hands work to the executor. Workers execute the tasks and write status back to the metadata database. The web UI reads the same database to show what happened.

Running Pipelines

Schedules, states, executors, and code pattern

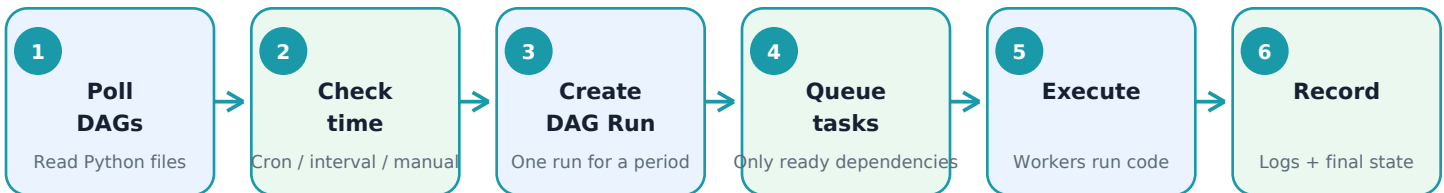


6 Task instance lifecycle



The scheduler decides the state. The worker does the work. The metadata database remembers everything.

7 How scheduling works



8 Minimal ETL DAG pattern

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract(): print("Pulling data...")
def transform(): print("Cleaning data...")
def load(): print("Writing to warehouse...")

with DAG("etl_pipeline", start_date=datetime(2024, 1, 1),
        schedule="@daily", catchup=False) as dag:
    t1 = PythonOperator(task_id="extract", python_callable=extract)
    t2 = PythonOperator(task_id="transform", python_callable=transform)
    t3 = PythonOperator(task_id="load", python_callable=load)
    t1 >> t2 >> t3 # left runs before right
```

9 Executor comparison

Sequential

One task at a time. Dev/testing only. Zero real parallelism.

Local

Parallel tasks on one machine. Good for smaller production setups.

Celery

Distributed workers via Redis/RabbitMQ. Classic multi-machine pattern.

Kubernetes

One pod per task. Isolated, scalable, cloud-native.

How to Think About Airflow

Use cases, boundaries, best practices, and interview language



10 Airflow is / Airflow is not

Airflow is

- A workflow orchestrator
- A scheduler for recurring pipelines
- A monitoring layer for pipeline runs
- A way to define pipelines as code

Airflow is not

- The main data processing engine
- A low-latency streaming platform
- A replacement for Spark, SQL engines, or warehouses
- A magic fix for messy data logic

11 Best use cases



ETL / ELT

Extract, transform, load on a schedule.



Warehouse

Daily marts, summaries, dashboards.



Batch ML

Features, train, evaluate, publish.



Quality

Row counts, null checks, anomaly checks.



Automation

Coordinate APIs, files, jobs, alerts.

12 Production habits that matter

- Keep DAG files declarative: describe the pipeline shape, not all heavy logic.
- Avoid heavy imports or database calls while the scheduler parses DAG files.
- Put reusable processing logic in packages, SQL files, Spark jobs, or containers.
- Pass small metadata with XCom, not large datasets.

Apache Airflow is a workflow orchestration tool. It lets us define pipelines as Python DAGs, schedule them, manage dependencies, retry failures, and monitor execution through the UI. The key point: Airflow orchestrates the work; Spark, SQL engines, Python scripts, cloud services, or warehouses usually do the heavy processing.

Quick takeaway: Airflow is the control tower for scheduled, dependency-driven data workflows.